Human-in-the-Loop Dynamic Pricing for Football Tickets

Tomás Ravalli*
ML Engineer · hello@0trm.dev

Abstract. Football clubs still price most match tickets once per season and update them by hand. This paper describes a system that recommends a daily price for every match and seating zone, with a pricing analyst approving each change. A Prophet and XGBoost ensemble forecasts demand, and a grid search picks the revenue-maximizing price among the prices each zone has actually been sold at, since tree models cannot extrapolate beyond them. In a 2023 deployment at a European football club, segment-based A/B tests showed **+6% revenue per match** and **+4% sell-through**, prices moved from weekly to daily updates, and analysts approved **86%** of proposals. On synthetic data with a known demand curve, recommendations earn 95% of the best achievable revenue.

1 INTRODUCTION

Ticket revenue is one of the few income lines a football club controls directly on match day, and it is usually priced with the least data. The common practice, which the club in this study followed, is to sort fixtures into a handful of categories before the season (for example A++, A and B), attach a price list per seating zone to each category, and revise prices weekly or monthly. Each revision meant pulling data by hand from several systems, proposing changes, and typing approved prices into the ticketing platform.

This leaves money on the table in both directions. Demand for a fixture moves with things a category set months earlier cannot see: the opponent's recent form, injuries to star players, whether the league title is already decided, the weather forecast, a concert in the city on the same night. Prices that are too low sell out early and push revenue to the secondary market; prices that are too high leave seats empty, which also costs in-stadium spending and atmosphere.

Dynamic pricing in professional sport is not new [2], and the revenue management toolkit behind it is mature [4]. The harder part in practice is getting a commercial team to trust and use a model's prices. This paper describes a system built around that constraint and reports what it delivered. Its contributions are:

1. **A forecast-then-optimize engine** that combines per-series Prophet models [5] with a pooled XGBoost residual model [1], plus a pooled fallback for fixtures with no sales history yet.
2. **A support-constrained price search.** Restricting the grid search to prices each zone was actually sold at raised the revenue share of recommendations from 26% to 95% on synthetic data.
3. **A human-in-the-loop workflow** in which analysts simulate, approve or reject each proposal, which reached 86% adoption.
4. **Deployment evidence** from segment-based A/B tests at a European football club, and a reimplementation on synthetic data.

*Work done in 2023 while at the club. The club approved this publication in writing and is not named.

2 PROBLEM SETTING

For each upcoming match m, seating zone z and day t before the match, the club sets a price p. Let $\hat{s}_{m,z,t}(p)$ be forecast ticket sales at that price given everything known at t. The engine recommends

$$p^* = \arg\max_{p \in \mathcal{P}_z} p \cdot \hat{s}_{m,z,t}(p), \tag{1}$$

where $\mathcal{P}_z$ is the set of candidate prices for zone z (Section 5). Three properties of the setting shape the design. First, each match is sold only once, over a window of about three months, so there is little history per series and none at all when a fixture goes on sale. Second, historical prices were not set at random: they track how attractive the match looked, so price and demand are confounded. Third, a wrong price is visible to fans and costly to the brand, so a person has to own every change.

3 SYSTEM OVERVIEW

Figure 1 shows the system in context. The engine reads historical batch data from the club's data systems and near real-time sales and availability from the ticketing platform. It sends price proposals to a pricing analyst, who simulates and approves them, and pushes approved prices back to the ticketing platform, where fans buy. New sales then feed the next forecast, closing the loop. The engine also reports model performance and drift metrics to the analyst.

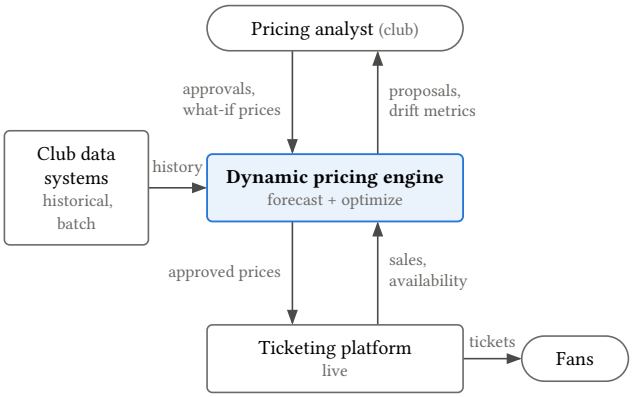


Figure 1: System context. The analyst approves every price before it reaches the live ticketing system.

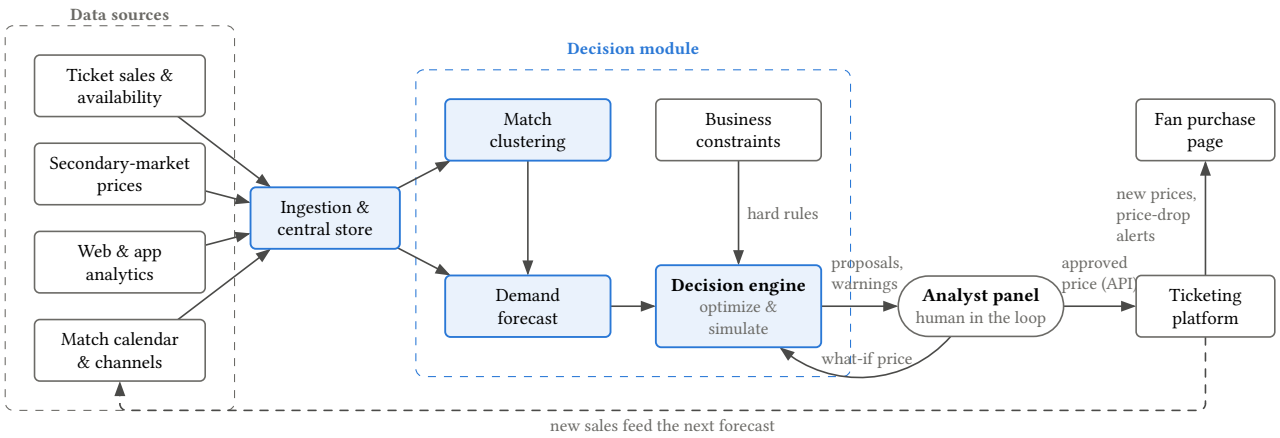


Figure 2: Architecture of the deployed engine. Blue: components of the engine; dashed line: the feedback loop from live sales to the next forecast.

Figure 2 shows the internal architecture. Four groups of sources feed a central data store: ticket sales and availability, competitor prices on secondary markets, web and app analytics (visits and conversion), and the match calendar with competitions and sales channels. The decision module contains the demand forecast model (Section 4), a match clustering step whose output feeds the forecast model, and the decision engine that optimizes and simulates prices (Section 5). Business constraints, such as price floors and ceilings set by the club, enter the decision engine as hard-coded rules. The outputs are price variation proposals, impact simulations and anomaly warnings, all shown in the analyst's control panel (Section 6). Approved prices go to the ticketing system through a REST API, and price drops also raise an alert on the fan-facing purchase page.

4 DEMAND FORECASTING

4.1 Features

Table 1 lists the features. Most are known well before the match; the demand signals (search trends, social media sentiment, web visits) are observed daily. One candidate feature, the web conversion rate, was deliberately excluded: it is defined as sales divided by web visits, so at decision time it contains the quantity being predicted. Keeping it inflates offline accuracy and breaks the price response the optimizer depends on.

Table 1: Feature set used by the demand model.

Group	Features
Match	days until match, weekday flag, opponent tier, opponent strength rating, international competition flag
Team status	league position, top player injured, league winner already known
Ticket & zone	seat zone, ticket price, zone seats available, share of tickets still available
Demand signals	internal site search volume, search engine trend index, social media sentiment, web visits
External	public holiday, major concert in the city, competitor average price, inbound flights index, weather forecast
Target	tickets sold per zone and day

4.2 Two-stage ensemble

The forecast has two stages. **Stage A** fits one Prophet model [5] per (match, zone) series, with weekly seasonality and weekday and holiday regressors. It captures the shape of a sales curve over the selling window, which is mostly driven by time to kick-off. **Stage B** fits a single XGBoost regressor [1], pooled across all series, on Stage A's in-sample residuals, using the full feature set including price. It learns the non-linear interactions Prophet cannot, such as how price sensitivity changes with the opponent or with days left. The forecast is

$$\hat{s} = \max\big(0, \hat{y}_{\text{Prophet}} + \hat{r}_{\text{XGB}}\big). \tag{2}$$

4.3 Cold start

A fixture that has just gone on sale has no sales history, so Stage A has no model for it. Without a fallback the forecast is the residual alone, which is meaningless. The engine therefore also trains a third model: an XGBoost regressor fit directly on sales with the same features, pooled across series. It serves any series without a Prophet model until enough history accumulates.

5 PRICE OPTIMIZATION

The engine solves Equation 1 by grid search. For each candidate price it builds a feature row that differs only in

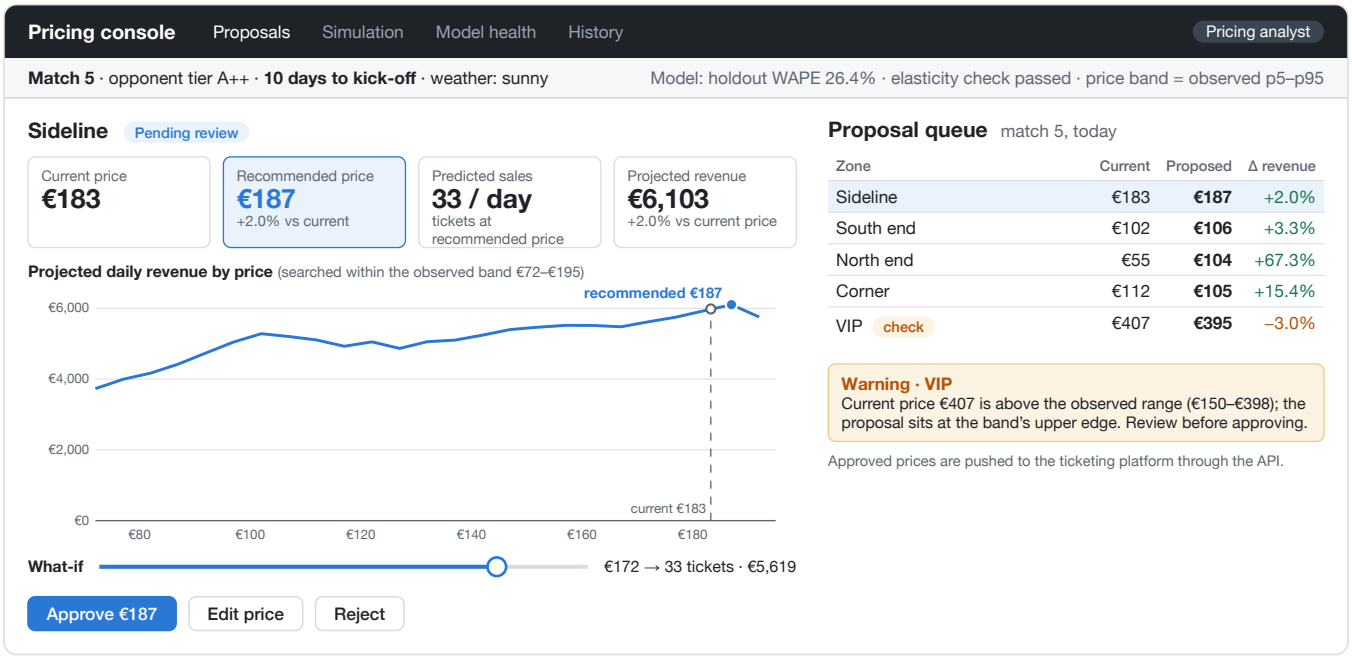


Figure 4: Mockup of the analyst control panel, filled with the reimplementation's proposals for one synthetic match ten days before kick-off: the selected zone's recommendation, its revenue curve within the observed price band, a what-if price, and the queue of proposals for the other zones with a warning raised by the engine.

price, predicts sales for the whole batch in one call, computes revenue as price times predicted sales, and returns the maximum. Grid search was chosen over Bayesian optimization because it guarantees the best price within the search space; for a decision that goes to customers, that guarantee is worth the modest extra compute of one prediction batch per match and zone.

Searching only observed prices. The most consequential design choice is the search space $\mathcal{P}_z$. Tree ensembles cannot extrapolate: above the highest price seen in training, XGBoost predicts flat sales, so revenue appears to keep rising with price and the argmax runs to whatever cap the grid allows (Figure 3). The engine therefore searches only between the 5th and 95th percentile of prices each zone was actually sold at, computed at training time, in €5 steps. On the synthetic data, no training row is priced above 1.9× base; searching the wider band of 0.5× to 2.5× base with the same model sends 72% of recommendations to the cap and earns 26% of the best achievable revenue, against 95% with the constrained band (Section 7.3).

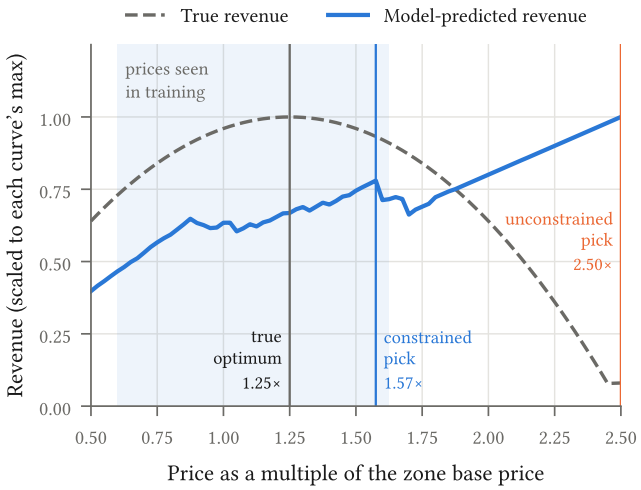


Figure 3: Revenue against price for the sideline zone, averaged over its holdout rows. Inside the observed band the model's curve peaks near the true optimum; beyond it the curve keeps rising, so an unconstrained search picks the cap.

Sanity checks. Before a model is used, an automated check samples historical rows, sweeps each one through its zone's price band and fails if predicted sales do not respond to price (less than 20% relative spread, or not monotone within two tickets per step) or if more than 40% of optimal prices sit at the band's upper edge.

6 HUMAN IN THE LOOP

Every recommendation becomes a price variation proposal for a pricing analyst. The control panel (Figure 4) shows, for a chosen match and zone, the current price, the recom-

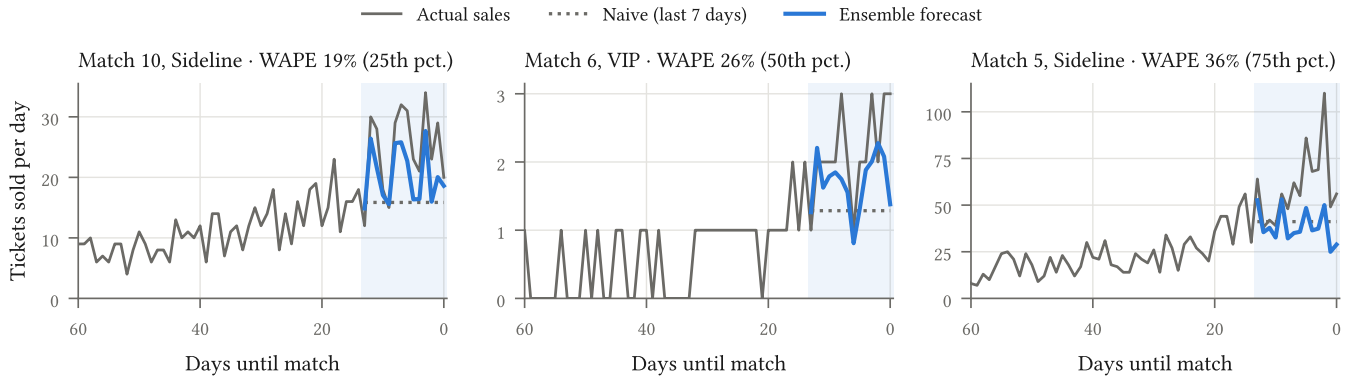


Figure 5: Ensemble forecasts on the 14-day holdout (shaded) for the series at the 25th, 50th and 75th percentile of holdout WAPE, with the naive forecast for reference.

mended price, predicted sales and projected revenue, and the full revenue-versus-price curve. A what-if slider lets the analyst try any other price and see its predicted sales and revenue before deciding. Approving sends the price to the ticketing platform through the API. The panel also surfaces anomaly warnings from the engine.

The point of this step is trust as much as safety. Analysts kept the final word, could see why a price was proposed, and could test alternatives in seconds. Pricing moved from a weekly or monthly manual exercise to a daily review of proposals.

7 OFFLINE EVALUATION ON SYNTHETIC DATA

The club's data cannot be shared, so the method is evaluated with a reimplementation on synthetic data built to mirror the structure of the real data. Its advantage is that the true price response is known, so recommendations can be scored against the best achievable revenue rather than only by forecast error.

7.1 Dataset and protocol

The generator produces 10 matches of varied attractiveness, each with a 91-day daily sales window, across up to five zones (VIP, sideline, corner and two end stands), with some zone-match pairs randomly dropped to mimic gaps in real data; 37 series and 3,367 zone-days remain. A single "match excitement factor" built from opponent tier, league position, injuries, match importance, holidays and weekday drives both the demand signals and the price level. Prices also carry wide operational variation (promotions, discounts, tests), which is what lets a model identify the price effect from history. Demand keeps a share $\max(0.02, 1 - r/2.5)$ of its potential at price ratio $r = p/p_{\text{base}}$, so revenue peaks at $r = 1.25$. All synthetic results below come from one run with pinned library versions (Python 3.12, macOS arm64); runs on Linux differ slightly.

The last 14 days of every series form the holdout. All models are retrained from scratch on the first 77 days, so no holdout information leaks into training. The main reference is a naive forecast anyone could make at the cutoff without a model: each series' mean daily sales over its last seven training days.

7.2 Forecast accuracy

Table 2 reports accuracy on the holdout. The ensemble's weighted absolute percentage error (WAPE) is 29% lower than the naive forecast. The XGBoost stage does most of the work: trained directly on sales it has the best WAPE and MAE, while the ensemble has the best RMSE and R^2, meaning fewer large misses. Prophet alone is worse than the naive forecast. Figure 5 shows three series chosen at the 25th, 50th and 75th percentile of holdout error, so the examples are typical rather than the best ones.

Table 2: Holdout forecast accuracy on synthetic data (last 14 days per series, tickets per zone-day).

Model	WAPE	R^2	MAE	RMSE
Ensemble (Prophet + XGBoost)	26.4%	**0.729**	6.2	**11.7**
Prophet only	41.8%	0.483	9.8	16.2
XGBoost only (on sales)	**22.6%**	0.717	**5.3**	12.0
Naive: last 7 days	37.4%	0.544	8.7	15.2
Naive: global mean	79.7%	−0.428	18.6	27.0

7.3 Decision quality

Forecast accuracy is not the goal; the price is. For every holdout row, the optimizer's recommended price is scored by the share of the best achievable revenue it earns under the true demand curve (Figure 6). Recommendations earn 95% on average, from 91% in VIP, the lowest-volume zone, to 98% in the corners. The median recommendation is 1.52× base price against a true optimum of 1.25×, and 19% of recommendations sit at the upper edge of the band: the model still underestimates how fast demand falls at high prices. With the unconstrained band the same model earns 26%.

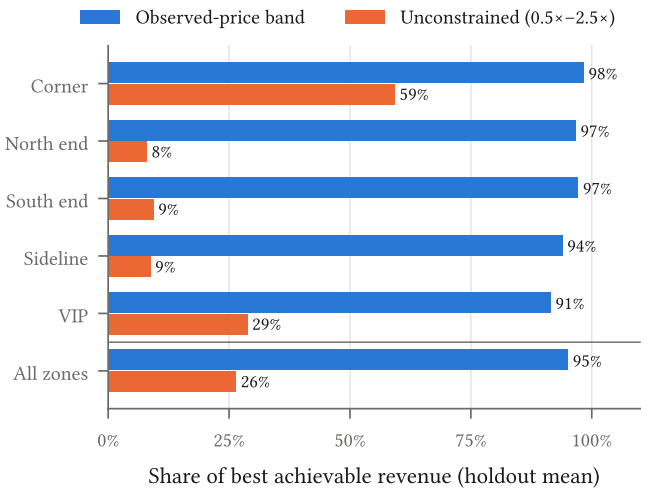


Figure 6: Share of the best achievable revenue earned by holdout recommendations, by zone, with the observed-price band and with the unconstrained band.

7.4 Cold start

To mimic a fixture that has just gone on sale, each match is left out of training in turn and forecast as unseen (Figure 7). Without a fallback the median WAPE across the 10 matches is 89.8%. The pooled XGBoost brings it to 12.5% (9.2% to 46.4% across matches), and its recommendations on unseen matches earn 94% of the best achievable revenue.

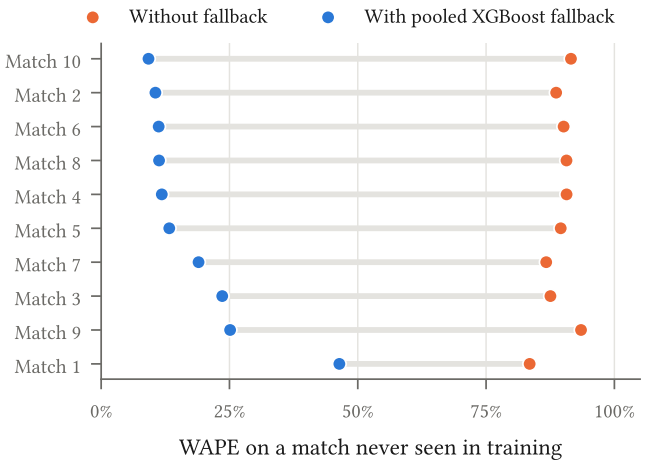


Figure 7: Forecast error on each match when it is left out of training, with and without the pooled fallback model.

8 DEPLOYMENT RESULTS

The engine was deployed at the club in 2023 and evaluated with segment-based A/B tests. A subset of seating zones was priced by the engine (treatment) while the remaining zones kept the static price list (control). Tests ran across matches of varying importance so that the lift could not come from a single high-demand fixture. Table 3 summarizes the results.

Table 3: Results of the 2023 deployment. These figures come from confidential club data and cannot be reproduced with the reimplementation on synthetic data.

Outcome	Result
Revenue per match (treatment vs. control)	**+6%**
Sell-through rate	**+4%**
Price update cadence	weekly → daily (7×)
Proposals approved by analysts	**86%**

Revenue and sell-through rose together. Had the gain come only from charging more, occupancy would have fallen; instead more seats were sold at prices closer to what each fixture could bear, which also benefits atmosphere and in-stadium spending. Automating data collection and analysis is what made daily updates possible. The 86% approval rate is the clearest sign that the commercial team trusted the proposals enough to use them, which was the original goal.

9 LESSONS LEARNED

Score the decision, not the forecast. The model with the best forecast metrics is not automatically the one that picks the best prices. Evaluating recommendations against a known demand curve exposed a failure that accuracy metrics hid completely.

Never let the optimizer search where the model has no data. Tree models predict flat demand outside the observed price range, so an unconstrained search will reliably recommend the maximum price. Constraining the search to observed prices was the largest single improvement in the reimplementation.

Remove features that leak the target. Conversion rate looked like a strong predictor and would have improved every offline metric while making the model useless for pricing.

Plan for fixtures with no history. Every match starts with zero sales. A pooled model that shares information across matches is required from day one, not an optional extra.

Keep a person in charge. A one-click approval with a transparent revenue curve and a what-if simulator turned a model into a tool the commercial team used daily.

Limitations. The deployment results come from one club and cannot be independently verified. Segment-based tests can be affected by fans switching between treated and control zones. The synthetic benchmark assumes a linear price response that does not change across matches, which is simpler than real demand.

10 CONCLUSION

A forecast-then-optimize engine with a human approval step moved a football club from seasonal, manual price lists to daily, data-driven prices, and delivered +6% revenue per match with +4% sell-through and 86% adoption. The method uses standard tools; what made it work was constraining the price search to observed data, evaluating decisions rather

than forecasts, and keeping analysts in control. Appendix A summarizes the model in the model card format of Mitchell et al. [3].

REFERENCES

[1] Tianqi Chen and Carlos Guestrin. 2016. XGBoost: A Scalable Tree Boosting System. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016. 785–794. https://doi.org/10.1145/2939672.2939785

[2] Joris Drayer and Stephen L. Shapiro. 2012. Dynamic Ticket Pricing in Sport: An Agenda for Research and Practice. *Sport Marketing Quarterly* 21, 3 (2012), 184–194.

[3] Margaret Mitchell, Simone Wu, Andrew Zaldivar, Parker Barnes, Lucy Vasserman, Ben Hutchinson, Elena Spitzer, Inioluwa Deborah Raji, and Timnit Gebru. 2019. Model Cards for Model Reporting. In *Proceedings of the Conference on Fairness, Accountability, and Transparency (FAT* '19)*, 2019. 220–229. https://doi.org/10.1145/3287560.3287596

[4] Kalyan T. Talluri and Garrett J. van Ryzin. 2004. *The Theory and Practice of Revenue Management*. Springer.

[5] Sean J. Taylor and Benjamin Letham. 2018. Forecasting at Scale. *The American Statistician* 72, 1 (2018), 37–45. https://doi.org/10.1080/00031305.2017.1380080

A MODEL CARD

Model	Two-stage demand forecast plus a price optimizer. Stage A: one Prophet model per match and zone (weekly seasonality, weekday and holiday regressors). Stage B: one XGBoost regressor on Stage A's residuals (400 trees, learning rate 0.05, depth 5, row and column subsampling 0.9). Cold start: one XGBoost regressor on sales with the same features. Optimizer: grid search in €5 steps between the 5th and 95th percentile of each zone's observed prices.
Version and owner	Deployed at a European football club in 2023; a reimplementation on synthetic data by the author, with pinned library versions.
Intended use	Daily price proposals per match and seating zone, reviewed and approved by a pricing analyst, plus what-if simulation of alternative prices. Primary users: the club's commercial and pricing team.
Out of scope	Setting live prices without human approval. Recommending prices outside the range each zone was sold at. Personalized prices for individual fans. Other venues, event types or seasons without retraining and re-validation.
Inputs	Aggregate match-, zone- and day-level data only (Table 1): match context, team status, ticket price and availability, demand signals and external factors. No personal data about individual fans. The web conversion rate is excluded because it leaks the target.
Training data	Deployment: the club's ticketing, web analytics and secondary-market data (confidential). Reimplementation: synthetic data, 10 matches, 37 match-zone series, 3,367 zone-days; the first 77 days of each series are used for training.
Evaluation data	The last 14 days of each series (518 zone-days), never seen in training, and leave-one-match-out for cold start.
Metrics	Forecast: WAPE, MAE, RMSE and R^2 against a last-7-days naive forecast. Decision: share of the best achievable revenue under the known synthetic demand curve. Deployment: revenue per match, sell-through rate, update cadence and analyst approval rate from segment-based A/B tests.
Results	Synthetic holdout: WAPE 26.4% (naive 37.4%); recommendations earn 95% of the best achievable revenue (91% to 98% by zone); unseen matches: WAPE 12.5% and 94% of the best achievable revenue. Deployment: +6% revenue per match, +4% sell-through, daily instead of weekly updates, 86% of proposals approved.
Factors	Performance varies by zone (lowest in low-volume VIP zones), by match attractiveness and by time to kick-off. Fixtures with no sales history depend entirely on the pooled fallback model.
Ethical considerations	Higher prices for high-demand matches can price out local and long-standing fans, and frequent price changes can feel unfair to buyers. Mitigations in this system: a person approves every price, prices stay within the observed range and the club's business rules, and the model uses no individual-level data. Clubs adopting it should also set explicit price caps for fan segments they want to protect.
Caveats	The synthetic demand curve is linear and the same for every match, which is simpler than real demand. The model still recommends prices above the true optimum on average (median 1.52× against 1.25× base). Tree models predict flat demand outside the observed price range, which is why the optimizer must stay inside it. Deployment results come from one club and cannot be independently reproduced.